

华为技术有限公司	AscendTransformerBoost	
	版本：V0.2	共45页

AscendTransformerBoost接口说明书

(初稿)

(仅供内部使用)

拟制:	_____	日期:	2023/09/16
审核:	_____	日期:	_____
批准:	_____	日期:	_____



华为技术有限公司

版权所有 侵权必究



这修订记录

日期	修订版本	修改描述	作者

目录

1	概述	5
2	总体架构	5
3	接口命名规范	5
4	设计要求	5
4.1	接口兼容性原则	5
4.2	接口完备性要求	6
4.3	接口安全性要求	6
4.4	接口可靠性要求	6
4.5	性能约束	6
5	接口描述	7
5.1	namespace	7
5.2	atb/types.h	7
5.2.1	基本数据结构和常量	7
5.2.2	Dims	8
5.2.3	TensorDesc	9
5.2.4	Tensor	9
5.2.5	VariantPack	10
5.2.6	ReshapeFunc	10
5.2.7	InferShapeFunc	11
5.2.8	Node	11
5.2.9	GraphParam	11
5.3	atb/svector.h	12
5.3.1	SVector	12
5.4	atb/context.h	14
5.4.1	Context	14
5.4.2	CreateContext	14
5.4.3	DestroyContext	14
5.5	atb/operation.h	15
5.5.1	Operation	15
5.5.2	CreateOperation	15
5.5.3	DestroyOperation	16
5.6	atb/infer_op_params.h	16
5.6.1	加速库原生Operation列表	16
5.6.2	QuantType	17
5.6.3	ActivationOperation	18

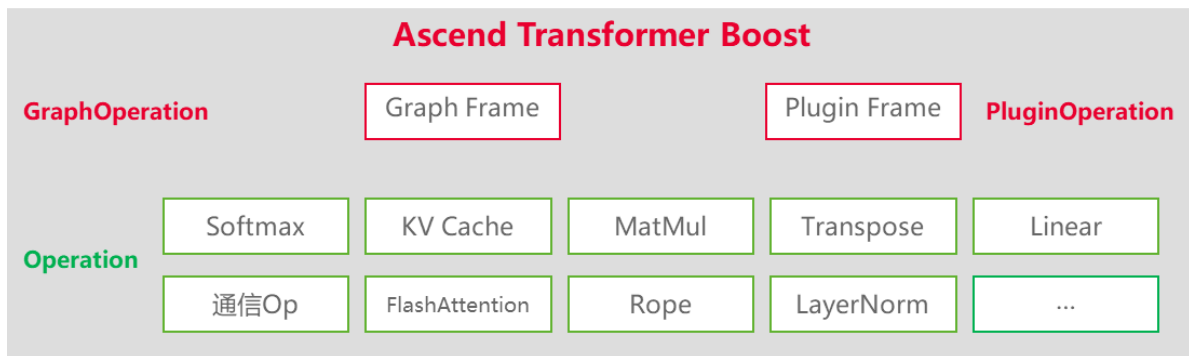
5.6.4	CumsumOperation	19
5.6.5	GatherOperation	20
5.6.6	MatmulOperation.....	21
5.6.7	MultinomialOperation	22
5.6.8	SplitOperation.....	23
5.6.9	ConcatOperation	24
5.6.10	SliceOperation	25
5.6.11	SoftmaxOperation.....	26
5.6.12	TransposeOperation.....	27
5.6.13	ElewiseOperation.....	27
5.6.14	FillOperation.....	31
5.6.15	LinearActivationOperation	32
5.6.16	LinearActivationQuantOperation	33
5.6.17	LayerNormOperation.....	34
5.6.18	RmsNormOperation.....	38
5.6.19	AllGatherOperation	41
5.6.20	AllReduceOperation	41
5.6.21	LinearOperation.....	42
5.6.22	LinearQuantOperation	44
5.6.23	LinearParallelOperation.....	45
5.6.24	RopeOperation.....	46
5.6.25	SelfAttentionOperation.....	47
5.6.26	KVCacheOperation	50
5.6.27	TransdataOperation	50
5.6.28	RepeatOperation	51
5.6.29	WhereOperation.....	52
5.6.30	SetValueOperation.....	53
5.6.31	Broadcast	54
5.7	atb/utls.h	55
5.7.1	Utils	55

1 概述

本文对AscendTransformerBoost加速库对外提供的接口，参数等进行了汇总描述，作为加速库用户和使用加速库进行模型加速设计的实现参考。

本文提供的接口，除特别说明外，310P/910A/910B芯片均支持。

2 总体架构



加速库接口功能主要分成三部分：

- 1) 提供基础原生的operation，用户可以创建使用对应的operation完成计算功能。
- 2) 提供plugin机制，用户可以根据自己的需求创建自定义的operation。
- 3) 提供算子图机制，用户根据模型使用加速库提供的原生operation和自定义operation创建算子图，加速相应的任务调度。

3 接口命名规范

通过namespace区分API类型，API的命名规范如下：

操作动词+对象

操作动词和对象均采用首字母大写。

4 设计要求

4.1 接口兼容性原则

【规则】：接口设计要具有抽象性，接口定义应该强调接口的逻辑语义而不是实现方法，以屏蔽掉具体实现的差异。

【示例】：昇腾加速库运行硬件环境有310/910等芯片，相同接口定义需要同时满足支持不同的硬件形态。

【规则】：系列化接口中所有版本应该保持后向兼容，在每个版本发布时提供兼容性列表。

【示例】：基于昇腾310芯片的系列化产品，加速库接口版本从1.0升级到X.Y($X, Y \geq 1$)，用户原来配套版本的应用软件无需升级。

【建议】：接口设计要具有良好的可扩展性，数据结构定义需要考虑预留字段以方便今后的扩展。

【规则】：接口废弃跟随客户市场需求变化，任何一个接口废弃必须通过评审确认。

4.2 接口完备性要求

【规则】：接口设计必须覆盖昇腾产品重点拓展行业典型应用场景。

【规则】：接口设计必须满足支撑典型应用场景的完整数据流。

4.3 接口安全性要求

【规则】：禁止接口参数传入可执行文件。

【规则】：内存操作类接口必须定义长度设置参数，限定内存资源访问的范围。

4.4 接口可靠性要求

【规则】：同步接口必须定义超时机制，合理设置超时时长，不能由于接口异常导致用户进程长时间挂死。

【建议】：执行时间较长的开放API接口应尽量采用异步调用获取操作结果，以避免系统阻塞。

【建议】：异步接口的任务执行建议定义超时机制，避免长时间占用设备侧资源。

4.5 性能约束

【规则】：新增特性或接口不能导致已有接口性能规格下降。

5 接口描述

5.1 namespace

atb

5.2 atb/types.h

5.2.1 基本数据结构和常量

```
typedef int32_t Status;
```

```
constexpr uint32_t MAX_DIM = 8;
```

当前所有接口tensor支持的最大维度为8

```
constexpr uint32_t IN_TENSOR_SARTD_ID          = 0;
constexpr uint32_t OUT_TENSOR_START_ID         = 0x10000;
constexpr uint32_t INTERNAL_TENSOR_START_ID    = 0x20000;
```

```
enum ErrorType : int {
    NO_ERROR = 0,
    ERROR_INVALID_PARAM, // 无效参数
    ERROR_INVALID_GRAPH, // 图错误
    ERROR_INTERNAL_ERROR,
    ERROR_RT_FAIL, // 调用 Runtime 接口失败
    ERROR_INVALID_IN_TENSOR_NUM, // 输入 Tensor 个数不一致
    ERROR_INVALID_TENSOR_DTYPE,
    ERROR_INVALID_TENSOR_FORMAT,
    ERROR_INVALID_TENSOR_DIM,
    ERROR_INVALID_TENSOR_SIZE,
    ERROR_OPERATION_NULL_RUNNER,
    ERROR_GRAPH_INFERSHAPE_FUNC_FAIL,
};
```

成员

成员名称	描述
NO_ERROR	正确
ERROR_INVALID_PARAM	无效参数
ERROR_INVALID_GRAPH	图错误
ERROR_RT_FAIL	调用runtime接口失败

ERROR_INVALID_IN_TENSOR_NUM	输入Tensor个数不一致
ERROR_INVALID_TENSOR_DTYPE,	DTYPE错误
ERROR_INVALID_TENSOR_FORMAT	FORMAT错误
ERROR_INVALID_TENSOR_DIM	DIM错误
ERROR_INVALID_TENSOR_SIZE	SIZE错误
ERROR_OPERATION_NULL_RUNNER	RUNNER空指针
ERROR_GRAPH_INFERSHAPE_FUNC_FAIL	INFERSHAPE错误

TensorId编码方案:

算子图的tensor编码需要按照该编码宏进行编码

样例参考

```
enum graphTensorID int {  
    inTensorId0      = ATB_IN_TENSOR_START_ID,  
    inTensorId1,  
    ...,  
    outTensorId0     = ATB_OUT_TENSOR_START_ID,  
    outTensorId1,  
    ...,  
    interTensorId0   = ATB_INTER_TENSOR_START_ID,  
    interTensorId1,  
    .../  
};
```

5.2.2 Dims

说明

Shape维度信息.

定义

```
struct Dims {  
    int64_t    dims[MAX_DIM];  
    uint64_t   dimNum = 0;  
};
```

成员

成员名称	描述
dims	每一维的大小
dimNum	总的维数, 最大支持8维

5.2.3 TensorDesc

说明

Tensor描述符, 描述每个tensor的数据类型, 格式和Dims信息

定义

```
struct TensorDesc {  
    aclDataType      dtype      = ACL_DT_UNDEFINED;  
    aclFormat        format     = ACL_FORMAT_UNDEFINED;  
    Dims             shape;  
};
```

成员

成员名称	描述
dtype	数据类型, 参考ACL定义的数据类型, 具体dtype格式说明, 请参考以下链接: https://www.hiascend.com/document/detail/en/canncommercial/601/inferapplicationdev/aclpythondev/aclpythondevg_01_0779.html
format	数据格式, 参考ACL定义的数据格式, 具体format格式说明, 请参考以下链接: https://www.hiascend.com/document/detail/en/canncommercial/601/inferapplicationdev/aclpythondev/aclpythondevg_01_0781.html
shape	Shape的Dims信息

5.2.4 Tensor

说明

Tesnor定义, 包含每个Tesnor的描述符, NPU内存地址, CPU内存地址, 内存大小等

定义

```
struct Tensor {  
    TensorDesc    desc;  
    void          *deviceData    = nullptr;  
    void          *hostData      = nullptr;  
    size_t        dataSize       = 0;  
};
```

成员

成员名称	描述
desc	Tensor描述符
deviceData	NPU tensor内存地址
hostData	CPU tensor内存地址
dataSize	内存大小

5.2.5 VariantPack

说明

输入输出Tensor打包结构

定义

```
struct VariantPack {  
    SVector<Tensor> inTensors;  
    SVector<Tensor> outTensors;  
};
```

成员

成员名称	描述
inTensors	输入tensor Vector
outTensors	输出tensor Vector

5.2.6 ReshapeFunc

说明

Reshape功能，改变tensor的shape

定义

```
using ReshapeFunc = std::function<void(const Dims &oldShape, Dims &newShape)>;
```

5.2.7 InferShapeFunc

说明

根据输入inTensor信息推导出输出outTensor信息

定义

```
using InferShapeFunc = std::function<Status(const SVector<TensorDesc> &inTensorDescs,  
SVector<TensorDesc> &outTensorDescs)>;
```

5.2.8 Node

说明

算子图中的Operation节点, 每个Node表示一个Operation或者GraphOperation. 所有的Node组成一个完整的算子图.

定义

```
struct Node {  
    Operation* operation = nullptr;  
    SVector<uint32_t> inTensorIds;  
    SVector<uint32_t> outTensorIds;  
    SVector<ReshapeFunc> inTensorReshapeFuncs;  
};
```

成员

成员名称	描述
operation	Node对应的operation或者graphOperation
inTensorIds	Node对应的operation或者graphOperation的输入tensorId Vector ^[注]
outTensorIds	Node对应的operation或者graphOperation的输出tensorId Vector ^[注]
inTensorReshapeFuncs	Node对应的operation或者graphOperation的每个输入tensor的reshape函数Vector ^[注]
注: 这三个成员均为Vector, 每个元素的顺序需要和对应tensor的顺序保持一致	

5.2.9 GraphParam

说明

算子图参数

定义

```
struct GraphParam {
    std::string          name; // 只允许字母、数字、下划线，长度不超过 128
    uint32_t             inTensorNum = 0;
    uint32_t             outTensorNum = 0;
    uint32_t             internalTensorNum = 0;
    std::vector<Node>    nodes;
    InferShapeFunc       inferShapeFunc;
};
```

成员

成员名称	描述
name	图name
inTensorNum	算子图输入tensor数量
outTensorNum	算子图输出tensor数量
internalTensorNum	算子图中间tensor数量
nodes	算子图Node Vector ^[注]
inferShapeFunc	inferShape函数指针
注: nodes的顺序, 需要满足各个node对应operation的执行顺序依赖关系, 先执行的在前面, 后执行的在后面	

5.3 atb/svector.h

5.3.1 SVector

说明

栈上的Vector, 提高访问效率, 使用时需要注意单个数据类型大小以及嵌套深度, 避免出现栈溢出的情况。

定义

```
template <class T, std::size_t MAX_SIZE = 48, bool CHECK_BOUND = true> class SVector {
public:
    constexpr SVector() : size_ { 0 } {}
    SVector(std::initializer_list<T> list)
    SVector(std::size_t size, const T &value = 0)
    void push_back(const T &val) noexcept(!CHECK_BOUND) && std::is_nothrow_assigna-
ble<T, const T &>::value)
    template <class... Args>
    void emplace_back(Args &&... args) noexcept(!CHECK_BOUND) && noexcept(Con-
structInPlace(std::declval<T *>(), std::forward<Args>(args)...))
    T *begin() noexcept
    const T *begin() const noexcept
    T *end() noexcept
    const T *end() const noexcept
    T *erase(const T *first, const T *last) noexcept(!CHECK_BOUND)
    T *erase(const T *at) noexcept(!CHECK_BOUND)
    std::size_t size() const noexcept
    T *insert(const T *pos, const T &value) noexcept(!CHECK_BOUND) &&
std::is_nothrow_assignable<T, const T &>::value)
    bool empty() const noexcept
    void clear() noexcept
    T *data() noexcept
    const T *data() const noexcept
    void resize(std::size_t size) noexcept(!CHECK_BOUND)
};
```

成员

成员名称	描述
SVector	构造函数
push_back	尾部添加一个元素
emplace_back	尾部添加一个元素
begin	返回首部地址
end	返回尾部地址
erase	删除某个位置元素
insert	在某个位置插入元素
empty	是否为空
clear	清空元素

data	返回数据地址
resize	改变数据大小

5.4 atb/context.h

5.4.1 Context

说明

Stream配置接口。

定义

```
class Context {  
public:  
    Context() = default;  
    virtual ~Context() = default;  
    virtual Status SetExecuteStream(aclrtStream stream) = 0;  
    virtual Status SetTilingCopyStream(aclrtStream stream) = 0;  
    virtual aclrtStream GetExecuteStream() const = 0;  
    virtual aclrtStream GetTilingCopyStream() const = 0;  
};
```

成员

成员名称	描述
SetExecuteStream	设置算子执行的运行Stream
SetTilingCopyStream	设置Tiling复制的Stream
GetExecuteStream	获取算子执行的Stream
GetTilingCopyStream	获取Tiling复制的Stream

5.4.2 CreateContext

说明

创建context接口。

定义

```
Status CreateContext(Context **context);
```

5.4.3 DestroyContext

说明

销毁context接口。

定义

```
Status DestroyContext(Context *context);
```

5.5 atb/operation.h

5.5.1 Operation

说明

Operation接口类。

定义

```
class Operation {
public:
    Operation() = default;
    virtual ~Operation() = default;
    virtual std::string GetName() const = 0;
    virtual Status InferShape(const SVector<TensorDesc> &inTensorDescs,
                              SVector<TensorDesc> &outTensorDescs) const = 0;
    virtual uint32_t GetInputNum() const = 0;
    virtual uint32_t GetOutputNum() const = 0;
    virtual Status Setup(const VariantPack &variantPack, uint64_t &workspaceSize) = 0;
    virtual Status Execute(const VariantPack &variantPack, uint8_t *workspace,
                           uint64_t workspaceSize, Context *context) = 0;
};
```

成员

成员名称	描述
GetName	获取op name
InferShape	根据输入inTensor信息推导出输出outTensor信息
GetInputNum	获取Op/GraphOp输入Tensor个数接口
GetOutputNum	获取Op/GraphOp输出Tensor个数接口
Setup	配置Op/GraphOp处理及获取workspace大小接口
Execute	Op/GraphOp执行接口

5.5.2 CreateOperation

函数功能

创建Operation/GraphOperation接口

函数原型

```
template <typename OpParam> Status CreateOperation(const OpParam &op-  
Param, Operation **operation);
```

参数说明

参数名称	输入/输出/返回	描述
Status	返回值	成功或者错误码
param	输入	Op/GraphOp的参数
operation	输出	返回创建的operation/graphOperation指针

5.5.3 DestroyOperation

函数功能

Op销毁接口

函数原型

```
Status DestroyOperation(Operation *operation);
```

参数说明

参数名称	输入/输出/返回	描述
Status	返回值	成功或者错误码
param	输入	Op/GraphOp指针

5.6 atb/infer_op_params.h

5.6.1 加速库原生 Operation 列表

序号	加速库原生Operation	说明
1	ActivationOperation	激活处理
2	AllGatherOperation	分布式多卡场景下综合处理
3	AllReduceOperation	
4	FillOperation	支持fill和markFill
5	TransposeOperation	转置处理

6	ConcatOperation	数据拼接
7	CumsumOperation	累积和计算
8	ElewiseOperation	ElementWise类计算
9	MatmulOperation	矩阵乘
10	TransdataOperation	数据格式转换
11	GatherOperation	数据抽取处理
12	KVCacheOperation	KVCache处理
13	LayerNormOperation	Norm处理
14	RmsNormOperation	
15	LinearOperation	Linear计算
16	LinearQuantOperation	
17	LinearParallelOperation	多卡并行Linear计算
18	MultinomialOperation	采样处理
19	RopeOperation	旋转位置编码
20	SelfAttentionOperation	Attention计算
21	SliceOperation	切片处理
22	SplitOperation	等分处理
23	SoftmaxOperation	SoftMax处理
24	LinearActivationOperation	Linear激活处理
25	LinearActivationQuantOperation	
26	RepeatOperation	维度大小扩展
27	WhereOperation	三目运算
28	SetValueOperation	setvalue拷贝
29	Broadcast	广播通信

5.6.2 QuantType

说明

量化类型枚举

定义

```
enum QuantType: int {  
    QUANT_UNDEINFED = 0,  
    QUANT_INT4,          // 当前不支持  
    QUANT_INT8,  
    QUANT_INT16,         // 当前不支持  
    QUANT_FLOAT8,        // 当前不支持  
    QUANT_FLOAT16,       // 当前不支持  
};
```

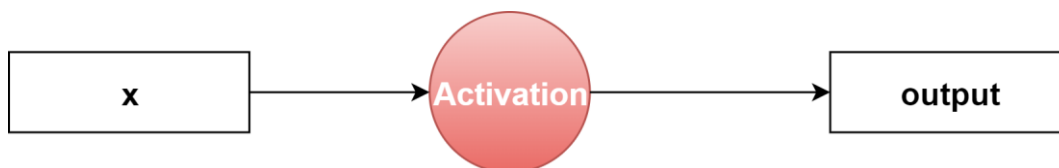
成员

成员名称	描述
QUANT_UNDEINFED	未定义, 默认不量化
QUANT_INT4	INT4量化, 当前不支持
QUANT_INT8	INT8量化
QUANT_INT16	INT16量化, 当前不支持
QUANT_FLOAT8	FLOAT8, 当前不支持
QUANT_FLOAT16	FLOAT16, 当前不支持

5.6.3 ActivationOperation

Operation功能

激活处理, 支持多种激活函数



约束

激活类型	数据类型
RELU	float
GELU	float/float16
FAST_GELU	float16
SWISH	float16
LOG	float/float16

Operation参数

定义

```
struct ActivationParam {  
    enum ActivationType: int {  
        ACTIVATION_UNDEFINED = 0,  
        ACTIVATION_RELU,  
        ACTIVATION_GELU,  
        ACTIVATION_FAST_GELU,  
        ACTIVATION_SWISH,  
        ACTIVATION_LOG,  
    };  
    ActivationType activationType;  
    float scale = 1.0f;  
};
```

成员

成员名称	描述
activationType	激活类型，支持RELU/GELU/FAST_GELU/SWISH/LOG
scale	SWISH激活函数的参数

Operation输入/输出

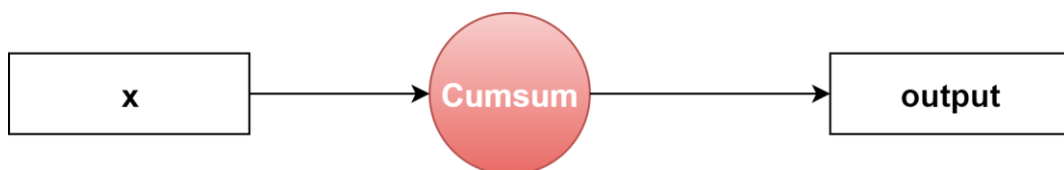
参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1]^{\text{[註]}}$	float16/float	ND	
output	OutTensor	$[-1, \dots, -1]$	float16/float	ND	输出和输入维度/类型/格式相同

注: -1表示当前维度的大小没有约束，本文后续维度中的-1意义一致。

5.6.4 CumsumOperation

Operation功能

后处理累积和计算。



约束

axes只能包含一个轴索引。

Operation参数

定义

```
struct CumsumParam {
    SVector<int64_t>    axes;
    bool                exclusive;
    bool                reverse;
};
```

成员

成员名称	描述
axes	指定axis轴(维度)上计算累加和，只能包含一个轴索引
exclusive	在某一个轴上的累加结果从第几个元素开始 0: 从第一个元素开始; 1: 从0开始
reverse	正向累加或逆向累加 0: 输出正向累加; 1: 输出逆向累加

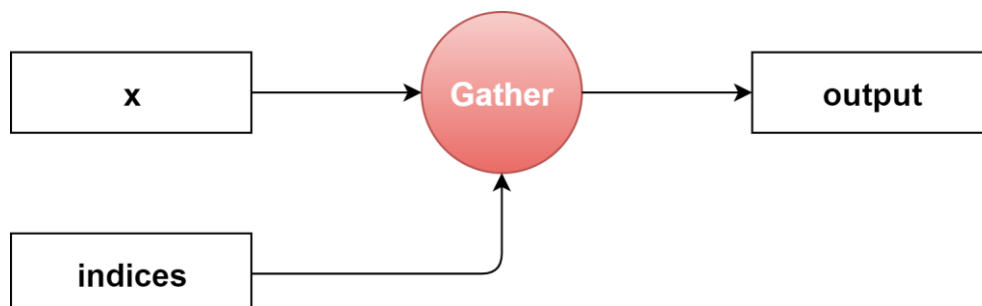
Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,..., -1]	float16	ND	
output	OutTensor	[-1,..., -1]	float16	ND	

5.6.5 GatherOperation

Operation功能

从inTensor中根据索引抽取指定的数据



约束

索引表, indices的元素大小小于指定轴(维度)的大小。

$x + \text{indices}$ 的维度相加 ≤ 9 。

Operation参数

定义

```
struct GatherParam {
    int    axis    = 0;
};
```

成员

成员名称	描述
axis	在指定axis轴(维度)上根据索引抽取指定元素

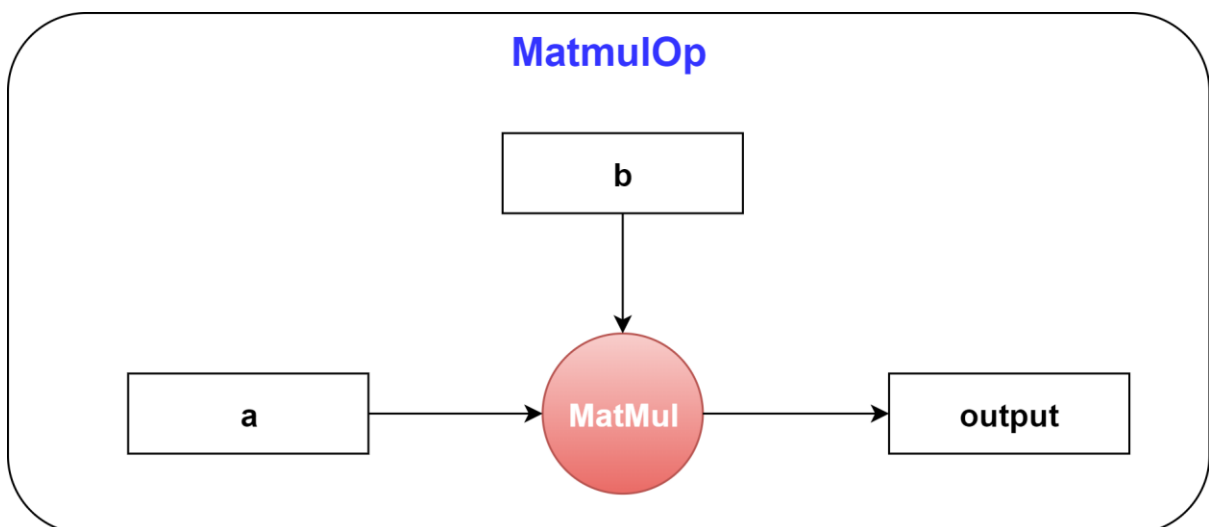
Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1]$	float16	ND	$x + \text{indices}$ 的维度相加 ≤ 9
indices	InTensor	$[-1, \dots, -1]$	int64	ND	索引表, indices的元素大小小于指定轴(维度)的大小
output	OutTensor	$[-1, \dots, -1]$	float16	ND	

5.6.6 MatmulOperation

Operation功能

矩阵乘



约束

x/y矩阵维度，通过transposeA/transposeB配置要满足矩阵乘的维度关系。

Operation参数

定义

```
struct MatmulParam {  
    bool transposeA = false;  
    bool transposeB = false;  
};
```

成员

成员名称	描述
transposeA	是否转置A矩阵
transposeB	是否转置B矩阵

Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[batch, m, k]	float16	ND	通过transposeA,transposeB保证满足矩阵乘的维度关系 y为NZ时，只有910A/310P支持，最后两维为16整数倍
y	InTensor	[batch, k, n]	float16	ND/NZ	
output	OutTensor	[batch, m, n]	float16	ND	

5.6.7 MultinomialOperation

Operation功能

采样功能。最后一个轴进行采样，随机抽取numSamples个值，输出下标。

随机数最多为64个。

约束

随机采样数numSamples要小于等于q对应的维度大小，最大为64。

最后一个轴要做归一化。

Operation参数

定义

```
struct MultinomialParam {  
    uint32_t numSamples = 1;  
};
```

成员

成员名称	描述
numSamples	采样数

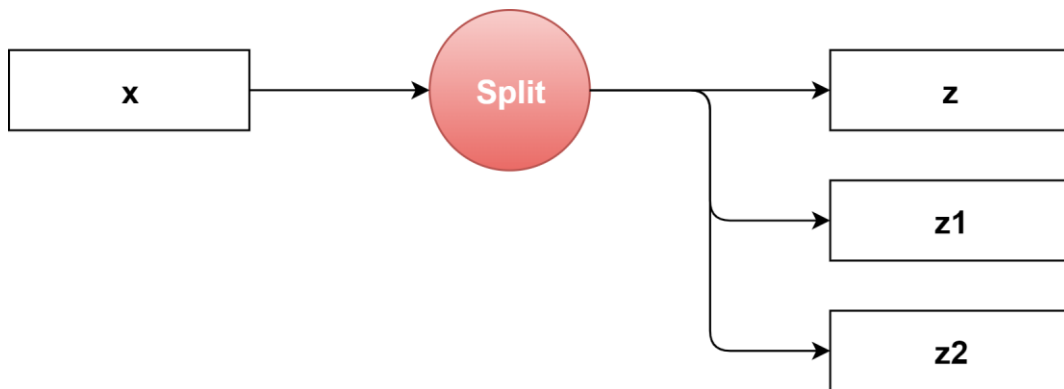
Operation输入/输出

参数	类型	维度	类型	格式	描述
q	InTensor	[batch, vocSize]	float16	ND	
output	OutTensor	[batch, numSamples]	uint32	ND	采数数据下标

5.6.8 SplitOperation

Operation功能

对splitDim维进行splitNum等分



约束

SplitNum支持2、3。SplitDim不能超过输入x的维度。

切分时，需要保证能够整除。

Operation参数

定义

```
struct SplitParam {
    int32_t    splitDim    = 0;
    int32_t    splitNum    = 2;
};
```

成员

成员名称	描述
splitDim	split的维度索引
splitNum	Split的等分次数, 当前支持2等分和3等分

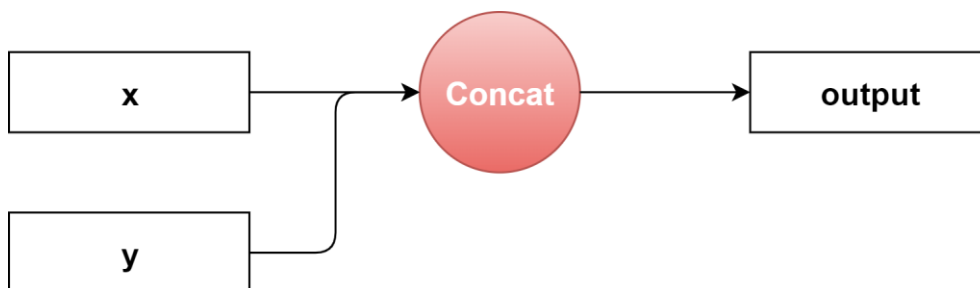
Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,...,-1]	float16/int64	ND	最高8维
z	OutTensor	[-1,...,-1]	float16/int64	ND	
z1	OutTensor	[-1,...,-1]	float16/int64	ND	
z2	OutTensor	[-1,...,-1]	float16	ND	splitNum=3时才有z2, 仅float16支持

5.6.9 ConcatOperation

Operation功能

Tensor拼接功能



约束

x/y的维度大小, 除了concatDim维外, 其他维度相同。

concatDim, 即拼接的维度, 要在x/y的维度范围内。

Operation参数

定义


```
struct ConcatParam {
    int    concatDim = 0;
};
```

成员

成员名称	描述
concatDim	指定拼接的维度

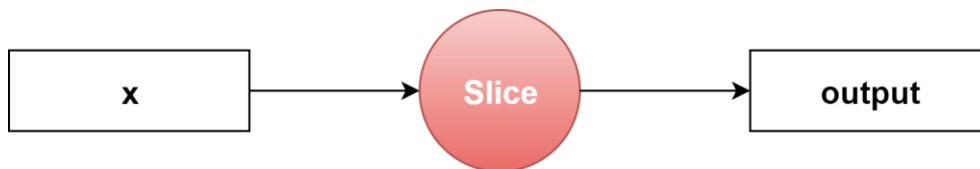
Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,...,-1]	float16	ND	
y	InTensor	[-1,...,-1]	float16	ND	除了concatDim维之外，其余维度y与x要相同
output	OutTensor	[-1,...,-1]	float16	ND	

5.6.10 SliceOperation

Operation功能

从tensor中提取切片，从offset位置开始，大小为size



约束

对应维度offsets，及offsets+size-1要在x的每个维度的大小范围内

Operation参数

定义

```
struct SliceParam {
    SVector<int64_t>    offsets;
    SVector<int64_t>    size;
};
```

成员

成员名称	描述
offsets	每个维度切片的起始位置，维度与输入Tensor一致
size	每个维度切片的大小，维度与输入Tensor一致

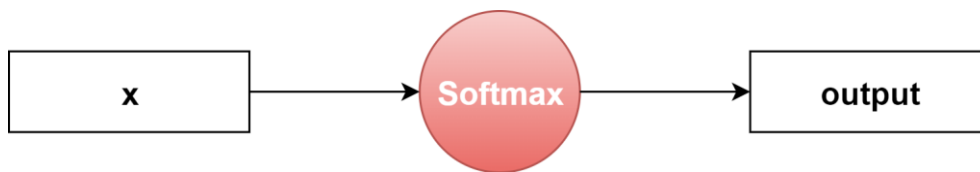
Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1]$	float16	ND	
output	OutTensor	$[-1, \dots, -1]$	float16	ND	每个维度的大小为size指定的大小

5.6.11 SoftmaxOperation

Operation功能

Softmax多分类激活函数，将多维（最大8维）Tensor数据在指定轴上映射到0到1之间，且非softmax轴数值之和为1



约束

axes可以支持多个轴上进行处理，最大维度为8。

axes不能为空，指定多个轴时，多个轴之间必须连续且从小到大。

Operation参数

定义

```

struct SoftmaxParam {
    SVector<int64_t>    axes;
};

```

成员

成员名称	描述
axes	指定轴(维度)

Operation输入/输出

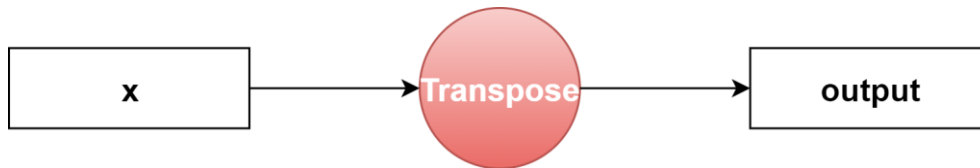
参数	类型	维度	类型	格式	描述
----	----	----	----	----	----

x	InTensor	[-1,...,-1]	float16/float	ND	
ouput	OutTensor	[-1,...,-1]	float16/float	ND	

5.6.12 TransposeOperation

Operation功能

改变输入Tensor的排列顺序，在多个维度上进行转置



约束

perm为输入维度的重排结果，需要保证输入正确，维度和x一致。

Opearation参数

定义

```

struct TransposeParam {
    SVector<int32_t>    perm;
};

```

成员

成员名称	描述
perm	指示输入维度的重排结果

Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,...,-1]	float16/int64	ND	
output	OutTensor	[-1,...,-1]	float16/int64	ND	

5.6.13 ElewiseOperation

Operation功能

Elewise计算功能

约束

ELEWISE_ADD/ELEWISE_MUL/ELEWISE_REALDIV将输入进行广播后再相加，x/y对应维度的对应值，需要满足，相同或者至少有一个是1。

Operation参数

定义

```
struct ElewiseParam {
    enum ElewiseType: int {
        ELEWISE_UNDEFINED = 0,
        ELEWISE_CAST,
        ELEWISE_MULS,
        ELEWISE_COS,
        ELEWISE_SIN,
        ELEWISE_NEG,
        ELEWISE_QUANT,
        ELEWISE_LOGICAL_NOT,
        ELEWISE_ADD,
        ELEWISE_MUL,
        ELEWISE_REALDIV,
        ELEWISE_LOGICAL_AND,
        ELEWISE_LOGICAL_OR,
        ELEWISE_LESS,
        ELEWISE_GREATER,
        ELEWISE_SUB,
        ELEWISE_EQUAL,
    };

    struct QuantParam {
        float inputScale = 1.0f;
        int inputOffset = 0;
    };

    struct MulsParam {
        float varAttr = 0.0f;
    };

    ElewiseType    elewiseType    = ELEWISE_UNDEFINED;
    QuantParam     quantParam;
    MulsParam      mulsParam;
};
```

成员

成员名称	描述
elewiseType	计算类型, 支持如下种类 ELEWISE_CAST: 数据类型转换 ELEWISE_MULS: 标量乘积 ELEWISE_COS: 余弦值计算 ELEWISE_SIN: 正弦值计算 ELEWISE_NEG: 取反 ELEWISE_QUANT: 量化, 当前只支持INT8 ELEWISE_LOGICAL_NOT: 逻辑取反 ELEWISE_ADD: 加, broadcast后再加 ELEWISE_MUL: 乘, broadcast后再乘 ELEWISE_REALDIV: 除, broadcast后再除 ELEWISE_LOGICAL_AND: 逻辑与 ELEWISE_LOGICAL_OR: 逻辑或 ELEWISE_LESS: 小于 ELEWISE_GREATER: 大于 EQUAL: 相等
quantParam	量化参数
MulsParam	Muls参数

ELEWISE_CAST输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,...,-1]	float16/float	ND	float<->float16
output	OutTensor	[-1,...,-1]	float/ float16	ND	

ELEWISE_MULS/ ELEWISE_COS/ ELEWISE_SIN输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,...,-1]	float/float16	ND	
output	OutTensor	[-1,...,-1]	float/ float16	ND	

ELEWISE_NEG输入输出

参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1]$	float16	ND	
output	OutTensor	$[-1, \dots, -1]$	float16	ND	

ELEWISE_QUANT输入输出

参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1]$	float16	ND	
output	OutTensor	$[-1, \dots, -1]$	int8	ND	

ELEWISE_LOGICAL_NOT输入输出

参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1]$	int8	ND	
output	OutTensor	$[-1, \dots, -1]$	int8	ND	

ELEWISE_ADD输入输出

参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1]$	float16/float	ND	
y	InTensor	$[-1, \dots, -1]$	float16/float	ND	
output	OutTensor	$[-1, \dots, -1]$	float16/float	ND	

ELEWISE_MUL输入输出

参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1]$	float16/float	ND	
y	InTensor	$[-1, \dots, -1]$	float16/float	ND	
output	OutTensor	$[-1, \dots, -1]$	float16/float	ND	

ELEWISE_REALDIV输入输出

参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1]$	float	ND	
y	InTensor	$[-1, \dots, -1]$	float	ND	
output	OutTensor	$[-1, \dots, -1]$	float	ND	

ELEWISE_LOGICAL_AND输入输出

参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1]$	int8	ND	
y	InTensor	$[-1, \dots, -1]$	int8	ND	

output	OutTensor	[-1,...,-1]	int8	ND	bool类型张量，维度为输入Broadcast后
--------	-----------	-------------	------	----	--------------------------

ELEWISE_LOGICAL_OR输入输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,...,-1]	int8	ND	
y	InTensor	[-1,...,-1]	int8	ND	
output	OutTensor	[-1,...,-1]	int8	ND	bool类型张量，维度为输入Broadcast后

ELEWISE_LESS/ELEWISE_GREATER/ELEWISE_EQUAL输入输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,...,-1]	int64/float/float16	ND	equal不支持int64
y	InTensor	[-1,...,-1]	int64/float/float16	ND	equal不支持int64
output	OutTensor	[-1,...,-1]	int8	ND	bool类型张量，维度为输入Broadcast后

5.6.14 FillOperation

Operation功能

将指定位置设置为value值，或者生成一个指定shape的Tensor，填充为value

约束

mask的元素只能是0或者1。mask可以被broadcast。

Operation参数

定义

```
struct FillParam {
    bool                withMask    = true;
    SVector<float>      value;
    SVector<int32_t>    outDim;
}
```

成员

成员名称	描述
withMask	是否Masked Fill
value	填充的元素
outDim	withMask=false时，输出tensor的shape

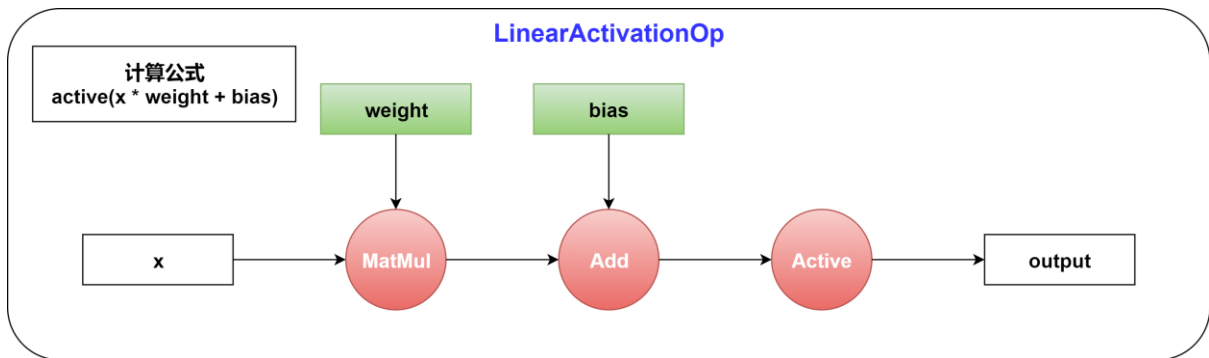
Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1]$	float16/int32	ND	withMask=true时输入
mask	InTensor	$[-1, \dots, -1]$	int8/bool	ND	withMask=true时输入
output	OutTensor	$[-1, \dots, -1]$	float16/int32	ND	输出的shape, withMask=true时和输入相同 withMask=false时由outDim指定

5.6.15 LinearActivationOperation

Operation功能

Linear激活处理



约束

x/weight矩阵维度, 通过transposeA/transposeB配置要满足矩阵乘的维度关系。

Operation参数

定义

```

struct LinearActivationParam {
    bool          transposeA          = false;
    bool          transposeB          = false;
    ActivationType activationFuncType = ACTIVATION_FAST_GELU;
    bool          hasBias              = true;
};

```

成员

成员名称	描述
transposeA	是否转置A矩阵

transposeB	是否不转置B矩阵
hasBias	是否叠加偏置
activationFuncType	激活函数类型

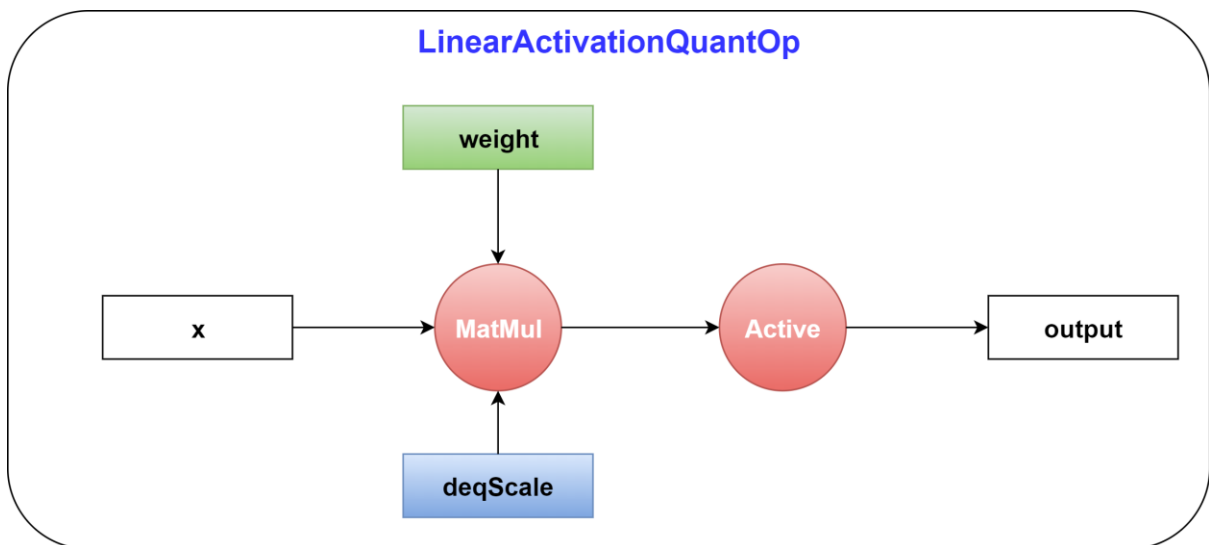
Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[batch, m, k]	float16	ND	
weight	InTensor	[k, n]	float16	ND	
bias	InTensor	[n]	float16	ND	hasBias=true时才输入
output	OutTensor	[batch, m,n]	float16	ND	

5.6.16 LinearActivationQuantOperation

Operation功能

量化Linear激活处理



约束

x/weight矩阵维度，通过transposeA/transposeB配置要满足矩阵乘的维度关系。

Operation参数

定义

```
struct LinearActivationQuantParam {
    bool transposeA = false;
    bool transposeB = false;
    ActivationType activationFuncType = ACTIVATION_FAST_GELU;
};
```

成员

成员名称	描述
transposeA	是否转置A矩阵
transposeB	是否不转置B矩阵
hasBias	是否叠加偏置
activationFuncType	激活函数类型

Operation输入/输出

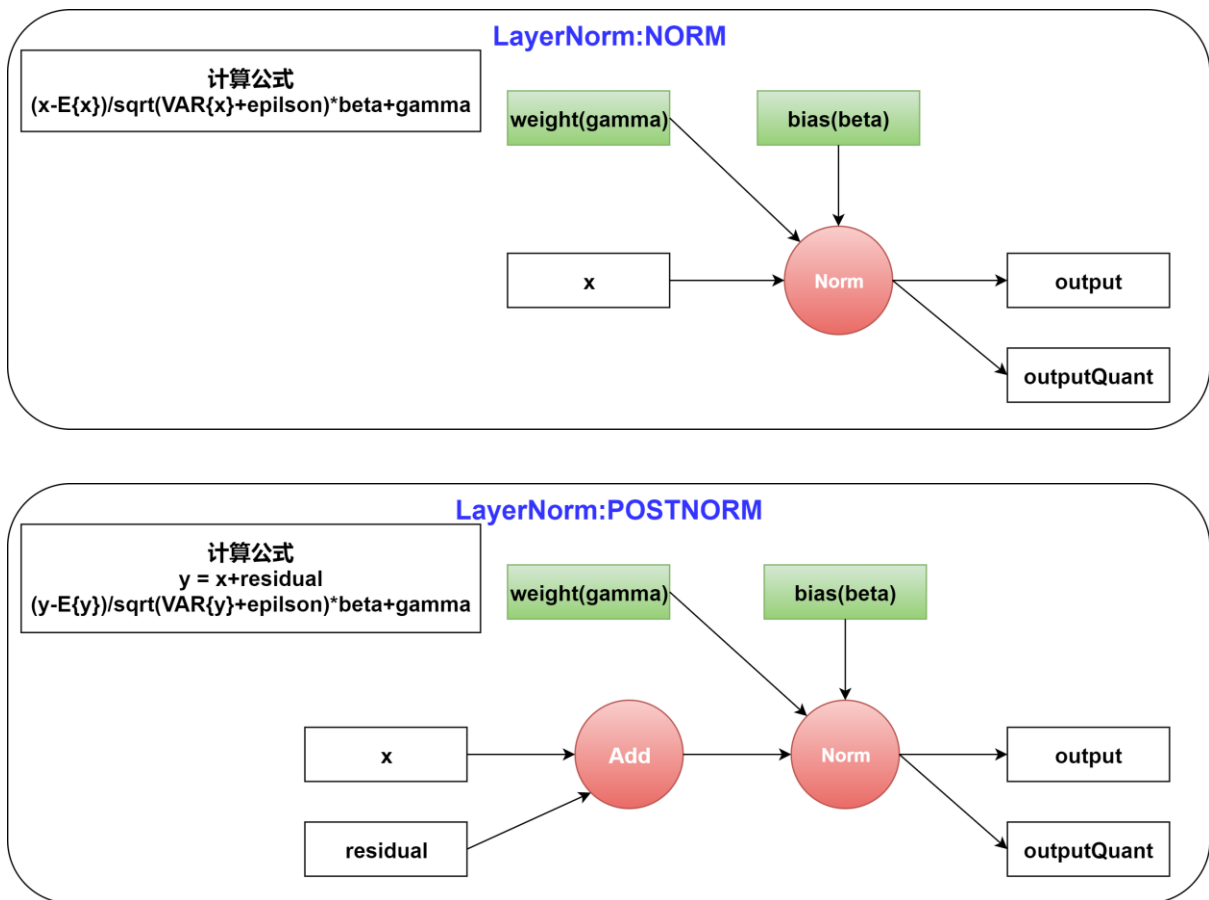
参数	类型	维度	类型	格式	描述
x	InTensor	[batch, m, k]	int8	ND	当前只支持int8
weight	InTensor	[k, n]	int8	ND	当前只支持int8
bias	InTensor	[1, n]	float16/int32	ND	hasBias=true时才输入 310P float16 910B int32
deqScale	InTensor	[1, n]	float32	ND	反量化的Scale 310P [batch, 1, n] 910B[batch, 1, 2n]
output	OutTensor	[batch, m,n]	float16	ND	

5.6.17 LayerNormOperation

Operation功能

归一化处理

当前支持两种, NORM/POSTNORM



约束

beginNormAxis维度小于等于最大维度

Opearation参数

定义

```
struct LayerNormParam {
    enum LayerNormType {
        LAYER_NORM_UNDEFINED = 0,
        LAYER_NORM_NORM,
        LAYER_NORM_PRENORM,
        LAYER_NORM_POSTNORM,
    };
    struct NormParam {
        QuantType quantType      = QUANT_UNDEINFED;
        float      epsilon       = 1e-5;
        int32_t    beginNormAxis  = 0;
        int32_t    beginParamsAxis = 0;
        float      quantInputScale = 1.0f;
        int        quantInputOffset = 0;
        float      quantInputAlpha = 1.0f;
    };
    struct PostNormParam {
        QuantType quantType      = QUANT_UNDEINFED;
        float      epsilon       = 1e-5;
        size_t     opMode        = 0;
        float      zoomScaleValue = 1.0f;
        float      quantInputScale = 1.0f;
        int        quantInputOffset = 0;
        float      quantInputAlpha = 1.0f;
    };

    LayerNormType layerType = LAYER_NORM_UNDEFINED;
    NormParam      normParam;
    PostNormParam  postNormParam;
};
```

成员

成员名称	描述
layerType	归一化类型, NORM/PRENORM(当前不支持)/POSTNORM 当前支持NORM和POSTNORM
normParam	NORM参数
postNormParam	POSTNORM参数

normParam成员

成员名称	描述
quantType	量化类型
epsilon	Epsilon, 归一化时加在分母上防止除零
beginNormAxis	归一化的维度 默认值为1, 从第几维开始norm, 同时决定输入gamma和beta维度
beginParamsAxis	
quantInputScale	量化参数, 非对称量化尺度变化 (调整fp的输出)
quantInputOffset	量化参数, 非对称量化偏移(输出后加上偏移)
quantInputAlpha	量化参数, 输出系数缩放因子

postNormParam成员

成员名称	描述
quantType	量化类型
epsilon	Epsilon, 归一化时加在分母上防止除零
opMode	0: 高精度; 1: 高性能
zoomScale	缩放因子
quantInputScale	量化参数, 非对称量化尺度变化 (调整fp的输出)
quantInputOffset	量化参数, 非对称量化偏移(输出后加上偏移)
quantInputAlpha	量化参数, 输出系数缩放因子

NORM输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,...,-1]	float16/float	ND	只有NORM支持float
gamma	InTensor	[beginNormAxis:] Or [1,-1]	float16/float	ND	NORM float16/float, 根据beginNormAxis确定维度其余均2维
beta	InTensor		float16/float	ND	
output	OutTensor	[-1,...,-1]	float16/float	ND	
outputQuant	OutTensor	[-1,...,-1]	int8	ND	quantType= QUANT_INT8时才输出

POSTNORM输入/输出

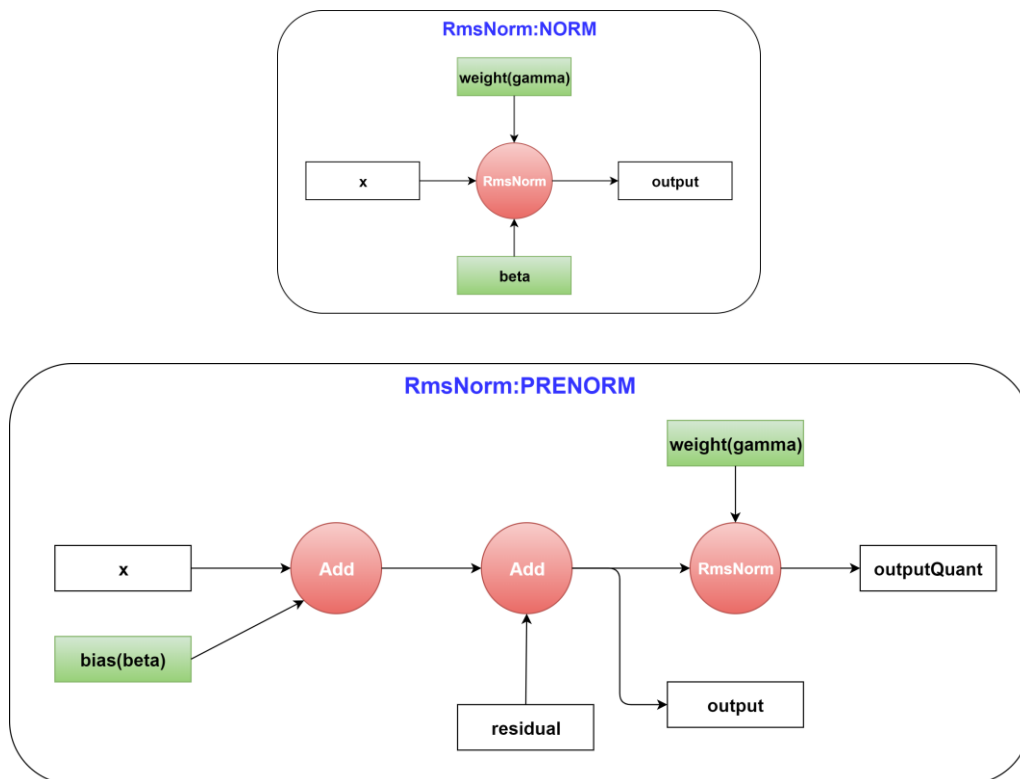
参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, n]$	float16	ND	
residual	InTensor	$[-1, \dots, n]$	float16	ND	
gamma	InTensor	$[1, n]$	float16	ND	最后一维的大小要32字节对齐
beta	InTensor	$[1, n]$	float16	ND	最后一维的大小要32字节对齐
output	OutTensor	$[-1, \dots, n]$	float16	ND	
outputQuant	OutTensor	$[-1, \dots, n]$	int8	ND	quantType= QUANT_INT8时才输出
所有tensor最后一维的n大小应该一样， $n \leq 6656$					

5.6.18 RmsNormOperation

Operation功能

RMS归一化处理

当前支持两种, NORM/PRENORM



Operation参数

定义

```
struct RmsNormParam {
    enum RmsNormType: int {
        RMS_NORM_UNDEFINED = 0,
        RMS_NORM_NORM,
        RMS_NORM_PRENORM,
        RMS_NORM_POSTNORM,
    };

    struct NormParam {
        QuantType    quantType        = QUANT_UNDEINFED;
        float        epsilon          = 1e-5;
        float        quantInputScale  = 1.0f;
        int          quantInputOffset = 0;
        double       layerNormEps     = 1e-5;
    };

    struct PreNormParam {
        QuantType    quantType        = QUANT_UNDEINFED;
        float        epsilon          = 1e-5;
        float        quantInputScale  = 1.0f;
        int          quantInputOffset = 0;
    };

    RmsNormType    layerType        = RMS_NORM_UNDEFINED;
    NormParam       normParam;
    PreNormParam    preNormParam;
};
```

成员

成员名称	描述
layerType	归一化类型, RMS_NORM_NORM/RMS_NORM_PRENORM/RMS_NORM_POSTNORM 当前支持RMS_NORM_NORM和RMS_NORM_PRENORM
normParam	NORM参数
preNormParam	PRENORM参数

normParam成员

成员名称	描述
------	----

quantType	量化类型
epsilon	Epsilon, 归一化时加在分母上防止除零
quantInputScale	量化参数, 非对称量化尺度变化 (调整fp的输出)
quantInputOffset	量化参数, 非对称量化偏移(输出后加上偏移)

preNormParam成员

成员名称	描述
quantType	量化类型
epsilon	Epsilon, 归一化时加在分母上防止除零
quantInputScale	量化参数, 非对称量化尺度变化 (调整fp的输出)
quantInputOffset	量化参数, 非对称量化偏移(输出后加上偏移)

RMS_NORM_NORM输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1, n]$	float16	ND	
gamma	InTensor	$[1, n]$	float16	ND	最后一维的大小要32字节对齐
beta	InTensor	$[1, n]$	float16	ND	quantType=INT8时输入 最后一维的大小要32字节对齐
output	OutTensor	$[-1, \dots, -1, n]$	float16/int8	ND	quantType=INT8时输出int8
所有tensor最后一维的n大小应该一样, $n \leq 8192$					

RMS_NORM_PRENORM输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	$[-1, \dots, -1, n]$	float16	ND	当前只支持quantType=INT8
residual	InTensor	$[-1, \dots, -1, n]$	float16	ND	
gamma	InTensor	$[1, n]$	float16	ND	最后一维的大小要32字节对齐
beta	InTensor	$[1, n]$	float16	ND	最后一维的大小要32字节对齐
output	OutTensor	$[-1, \dots, -1, n]$	float16	ND	
outputQuant	OutTensor	$[-1, \dots, -1, n]$	int8	ND	quantType=INT8时输出int8
所有tensor最后一维的n大小应该一样, $n \leq 8192$					

5.6.19 AllGatherOperation

Operation功能

把不同的卡的数据Concat起来

约束

rankSize不能为0, rank<rankSize, rankRoot<rankSize

Operation参数

定义

```
struct AllGatherParam {  
    int    rank          = 0;  
    int    rankSize      = 0;  
    int    rankRoot      = 0;  
    std::string backend  = "hccl";  
    void    *hcclComm    = nullptr;  
};
```

成员

成员名称	描述
rank	每个进程的编号
rankSize	总的进程数
rankRoot	主进程编号
backend	通信后端指示
hcclComm	hccl通信域接口获取的地址指针

Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,...,-1]	float16/float/int8/int32	ND	维度小于8
output	OutTensor	[rankSize, -1,...,-1]	float16/float/int8/int32	ND	维度小于等于8

5.6.20 AllReduceOperation

Operation功能

把不同的卡的数据相加或者平均

约束

rankSize不能为0，rank<rankSize, rankRoot<rankSize

Operation参数

定义

```
struct AllReduceParam {  
    int      rank          = 0;  
    int      rankSize      = 0;  
    int      rankRoot      = 0;  
    std::string allReduceType = "sum";  
    std::string backend      = "hccl";  
    void      *hcclComm     = nullptr;  
};
```

成员

成员名称	描述
rank	每个进程的编号
rankSize	总的进程数
rankRoot	主进程编号
allReduceType	reduceType指示
backend	通信后端指示
hcclComm	hccl通信域接口获取的地址指针

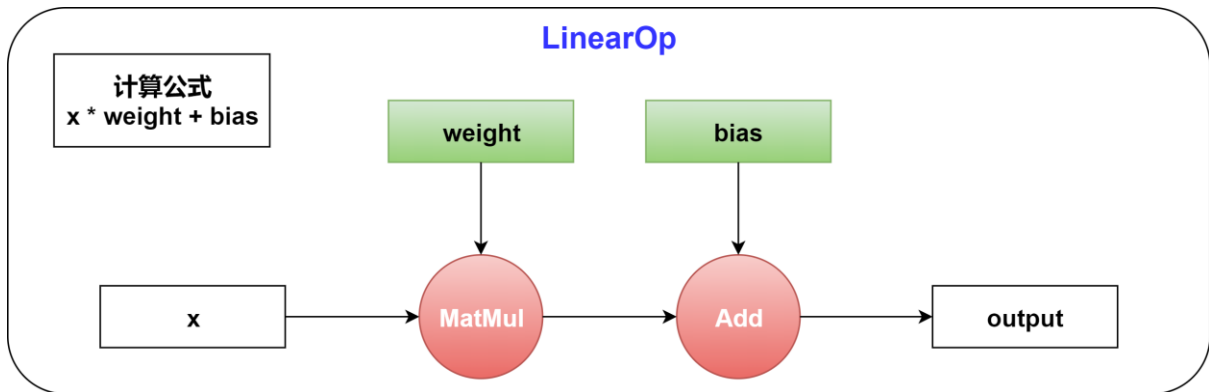
Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,...,-1]	float16/float/int8/int32	ND	
output	OutTensor	[-1,...,-1]	float16/float/int8/int32	ND	

5.6.21 LinearOperation

Operation功能

Linear计算功能



约束

x/weight矩阵维度，通过transposeA/transposeB配置要满足矩阵乘的维度关系。

Operation参数

定义

```

struct LinearParam {
    bool      transposeA    = false;
    bool      transposeB    = false;
    bool      hasBias       = true;
};
    
```

成员

成员名称	描述
transposeA	是否转置A矩阵
transposeB	是否不转置B矩阵
hasBias	是否叠加偏置

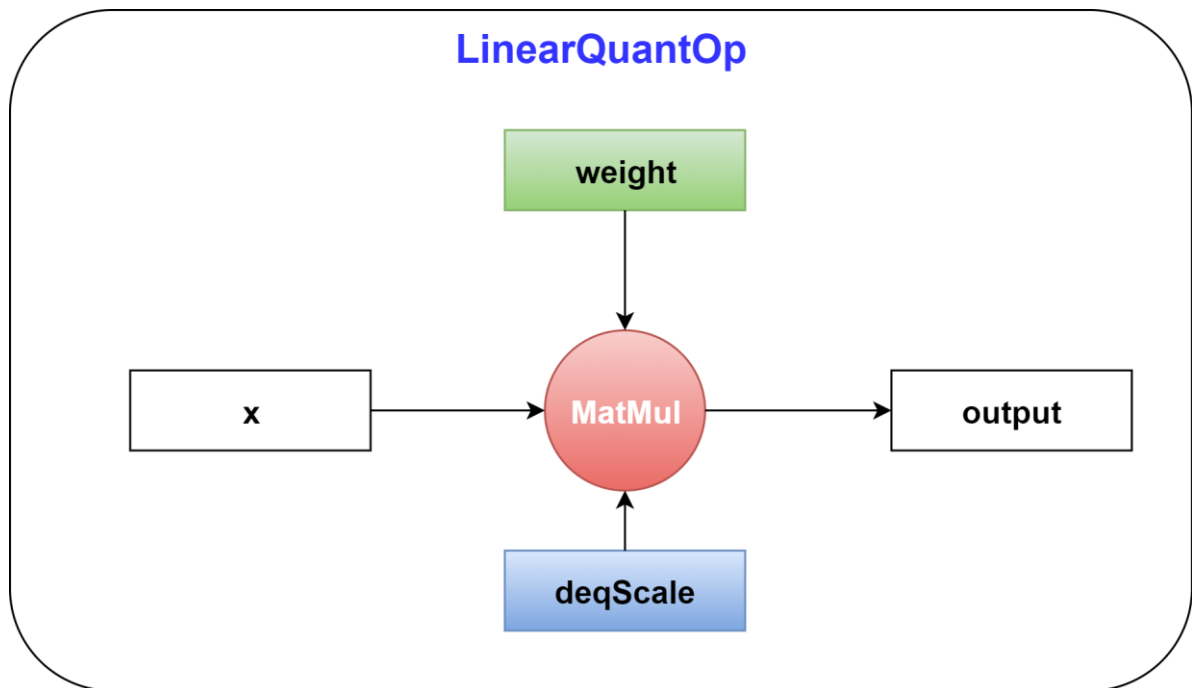
Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[batch, m, k]	float16	ND	
weight	InTensor	[k, n]	float16	ND/NZ	weight为NZ时，最后两维大小为16整数倍，只有910A/310P支持
bias	InTensor	[1, n]	float16	ND	hasBias=true时才输入
output	OutTensor	[batch, m,n]	float16	ND	

5.6.22 LinearQuantOperation

Operation功能

带量化的Linear



约束

x/weight矩阵维度，通过transposeA/transposeB配置要满足矩阵乘的维度关系。

Operation参数

定义

```
struct LinearQuantParam{
    bool        transposeA    = false;
    bool        transposeB    = false;
    bool        hasBias       = true;
};
```

成员

成员名称	描述
transposeA	是否转置A矩阵
transposeB	是否转置B矩阵
hasBias	是否叠加偏置

Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[batch, m, k]	int8	ND	当前只支持int8
weight	InTensor	[k, n]	int8	ND/NZ	当前只支持int8 weight为NZ时，最后一维大小为16整数倍，倒数第二维为32整数倍， 只有910A/310P支持
bias	InTensor	[1, n]	float16/int32	ND	hasBias=true时才输入 310P float16 910B int32
deqScale	InTensor	[1, n]	float32	ND	反量化的Scale 310P [batch, 1, n] 910B[batch, 1, 2n]
output	OutTensor	[batch, m,n]	float16	ND	

5.6.23 LinearParallelOperation

Operation功能

多卡并行Linear计算处理

约束

x/weight矩阵维度，通过transWeightt配置要满足矩阵乘的维度关系。

rankSize不能为0，rank<rankSize, rankRoot<rankSize

Operation参数

定义

```
struct LinearParallelParam {  
    bool            transWeight    = false;  
    int             rank           = 0;  
    int             rankSize       = 0;  
    int             rankRoot       = 0;  
    std::string     bias           = "";  
    std::string     parallelType   = "RowParallel";  
    std::string     backend        = "hccl";  
    void            *hcclComm      = nullptr;  
};
```

成员

成员名称	描述
transWeight	权重是否不需要转置
rank	每个进程的编号
rankSize	总的进程数
rankRoot	主进程编号
bias	
parallelType	权重并行方式
backend	通信后端指示
hcclComm	hccl通信域接口获取的地址指针

Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[batch, m, k]	float16	ND	
weight	InTensor	[k, n]	float16	ND	
bias	InTensor	[1, n]	float16	ND	bias非空时输入
output	OutTensor	[batch, m,n]	float16	ND	

5.6.24 RopeOperation

Operation功能

旋转位置编码处理。

约束

rotaryCoeff支持 $2/4/(\text{headDim}/2)$ 。

hiddenSizeQ是hiddenSizeK的整数倍， $\text{hiddenSizeQ}=\text{headDim}*\text{headNum}$ ，headNum为16倍数。

Operation参数

定义

```
struct RopeParam {  
    int32_t    rotaryCoeff    = 4;  
};
```

成员

成员名称	描述
rotaryCoeff	rope，旋转系数，对半旋转是2，支持配置 $2/4/(\text{headDim}/2)$

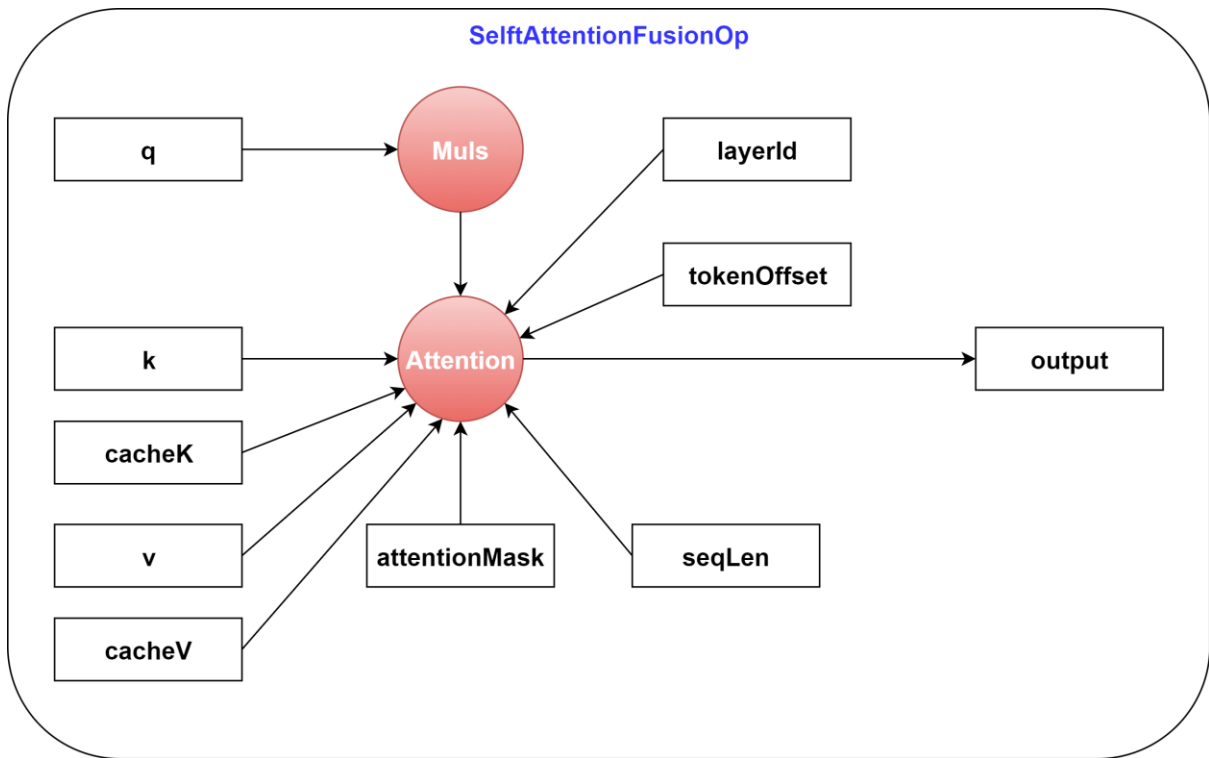
Operation输入/输出

参数	类型	维度	类型	格式	描述
q	InTensor	[ntokens, hiddenSizeQ]	float16	ND	hiddenSizeQ是hiddenSizeK的整数倍 $\text{hiddenSizeQ}=\text{headDim}*\text{headNum}$ ，headNum为16倍数
k	InTensor	[ntokens, hiddenSizeK]	float16	ND	
cos	InTensor	[ntokens, headDim]	float16	ND	
sin	InTensor	[ntokens, headDim]	float16	ND	
seqLen	InTensor	[batch]	uint32/int32	ND	
ropeQ	OutTensor	[ntokens, hiddenSizeQ]	float16	ND	
ropeK	OutTensor	[ntokens, hiddenSizeK]	float16	ND	

5.6.25 SelfAttentionOperation

Operation功能

完成Attention计算功能



约束

attentionMask的维度:

支持[batch, maxSeqLen, maxSeqLen], 此时需要保证batch的维度和其他输入batch维度的一致性。

支持[maxSeqLen, maxSeqLen], 此时所有batch的mask相同。

支持[batch, 1, maxSeqLen], 此时所有batch的mask不同, mask为向量。

tokenOffset >= seqLen

Operation参数

定义

```
struct SelfAttentionParam {
    int32_t          headDim          = 0;
    int32_t          headNum          = 0;
    float           qScale            = 1;
    float           qkScale           = 1;
    bool            isFusion          = true;
    bool            withCache         = true;
    bool            batchRunStatusEnable = false;
    int32_t          kvHeadNum        = 0;
};
```


成员

成员名称	描述
headDim	头维度
headNum	多头数量
qScale	q缩放系数
qkScale	qk缩放系数
isFusion	是否使用融合算子，当前只支持true
withCache	仅当isFusion为false时有用，intensor有pastKVCache
batchRunStatusEnable	是否动态batch
kvHeadNum	kvHeadNum=0时，kv的HeadNum和Q一致，是HeadNum kvHeadNum!=0时，kv的HeadNum为kvHeadNum

Operation输入/输出

参数	类型	维度	类型	格式	描述
q	InTensor	[nTokens, qHiddenSize]	float16	ND	nTokens约束batch间紧排
k	InTensor	[nTokens, hiddenSize]	float16	ND	
v	InTensor	[nTokens, hiddenSize]	float16	ND	
cacheK	InTensor	[layer, batch, maxSeqLen, hiddenSize]	float16	ND	
cacheV	InTensor	[layer, batch, maxSeqLen, hiddenSize]	float16	ND	
attentionMask	InTensor	[maxSeqLen, maxSeqLen] [batch, maxSeqLen, maxSeqLen] [batch, 1, maxSeqLen] [batch, headNum, maxSeqLen, maxSeqLen]	float16	ND	支持场景: 1) 所有batch相同，方阵 2) batch不同，方阵 3) batch不同，向量 4) 4维，支持alibi场景
tokenOffset	InTensor	[batch]	uint32	ND	计算完成后的tokenOffset device and host tensor
seqLen	InTensor	[batch]	uint32	ND	device and host tensor
layerId	InTensor	[1]	uint32	ND	

batchStatus	InTensor	[batch]	uint32	ND	batchRunStatusEnable=true时, host tensor
output	OutTensor	[ntokens, qHiddenSize]	float16	ND	

5.6.26 KVCacheOperation

Operation功能

KVCache处理

Operation参数

定义

```
struct KVCacheParam { };
```

成员

成员名称	描述
无	

Operation输入/输出

参数	类型	维度	类型	格式	描述
new_kv	InTensor	[ntokens, hiddenSize]	float16	ND	
layerId	InTensor	[1]	uint32	ND	
past	InTensor	[layer, batch, maxSeqLen, hiddenSize]	float16	ND	
tokenOffset	InTensor	[batch]	uint32	ND	
seqLen	InTensor	[batch]	uint32	ND	
present	OutTensor	[layer, batch, maxSeqLen, hiddenSize]	float16	ND	

5.6.27 TransdataOperation

Operation功能

数据格式转换处理

约束

使用的NZ的dims约定表示方式: {b, n1, m1m0, n0}, 对应的ND的dims是{b, m, n}, 其中:

b表示batch, 如果batch为1, 该维度为1, 不可省略; 如果batch有多个, 该维度为所有batch维度合轴的结果;

m0/n0表示对齐位，float16为16；

m1m0表示原始ND的m维度经过对齐位向上对齐；

n1表示原始ND的n维度经过对齐位向上对齐后，除以n0的商。

例如原始ND的dims为{8, 100, 30}，则其对应的NZ的dims为{8, 2, 112, 16}

Operation参数

定义

```
struct TransdataParam {
    enum TransdataType : int {
        UNDEFINED = 0,
        FRACTAL_NZ_TO_ND,
        ND_TO_FRACTAL_NZ
    };

    TransdataType      transdataType = UNDEFINED;
    SVector<int64_t> outCrops      = {0, 0};
};
```

成员

成员名称	描述
transdataType	数据格式转换类型，支持FRACTAL_NZ和ND互相转换
outCrops	FRACTAL_NZ转ND时，原ND数据格式shape的最后两维

Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[batch, m, n]	float16/int8	ND	
y	OutTensor	[batch, n1, m1m0, n0]	float16/int8	NZ	

参数	类型	维度	类型	格式	描述
x	OutTensor	[batch, n1, m1m0, n0]	float16	NZ	
y	InTensor	[batch, m, n]	float16	ND	

5.6.28 RepeatOperation

Operation功能

将输入tensor的shape，指定轴扩展到指定的倍数

约束

multiples要扩展的维度和当前x的维度总数小于等于8。

每一维上扩展的倍数>0。

multiples的维度要大于等于x的维度。

输出的维度和multiples维度一致；每个维度大小为x broadcast到multiples维度后和multiples对应维度的乘积。

Operation参数

定义

```
struct RepeatParam {
    SVector<int64_t>    multiples;
};
```

成员

成员名称	描述
multiples	每一维度上扩展的倍数 约束：要扩展的维度和当前的维度总数小于等于8。 每一维上扩展的倍数>0。 multiples的维度要大于等于x的维度。

Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,...,-1]	float16	ND	
y	OutTensor	[-1,...,-1]	float16	ND	输出的维度和multiples维度一致；每个维度大小为x broadcast到multiples维度后和multiples对应维度的乘积。

5.6.29 WhereOperation

Operation功能

三目运算operation, $z = \text{cond} ? x : y;$

约束

无

Operation参数

定义

```
struct WhereParam {  
};
```

成员

成员名称	描述
无	

Operation输入/输出

参数	类型	维度	类型	格式	描述
cond	InTensor	$[-1, \dots, -1]$	int8	ND	
x	InTensor	$[-1, \dots, -1]$	float16/float	ND	
y	InTensor	$[-1, \dots, -1]$	float16/float	ND	
z	OutTensor	$[-1, \dots, -1]$	float16/float	ND	

5.6.30 SetValueOperation

Operation功能

setvalue拷贝功能

dst: (a,b,c), src: (d,e,f)

dst[starts[0]: ends[0], starts[1]: ends[1], starts[2]: ends[2]] = src

约束

strides为全1。

dst每个维度的大小，小于等于src对应维度的大小。

src和dst的shape只有一个维度大小可以不同。如果有两个维度的大小不同，其中一个不同的维度必须是最高维。

Operation参数

定义

```
struct SetValueParam {  
    SVector<int64_t>    starts;  
    SVector<int64_t>    ends;  
    SVector<int64_t>    strides;  
};
```

成员

成员名称	描述
starts	每个轴的start
ends	每个轴的end
strides	当前只支持strides为全1

Operation输入/输出

参数	类型	维度	类型	格式	描述
src	InTensor	[-1,...,-1]	float16/float/int32/int64	ND	
dst	InTensor	[-1,...,-1]	float16/float/int32/int64	ND	setvalue结果

5.6.31 Broadcast

Operation功能

把主进程(root节点)的tensor数据广播到其他的进程。

Operation参数

定义

```
struct BroadcastParam {  
    int    rank = 0;  
    int    rankSize = 0;  
    int    rankRoot = 0;  
    HcclComm hcclComm = nullptr; // only effect when hcclComm is not null  
};
```

成员

成员名称	描述
rank	当前进程的编号
rankSize	总的进程数

rankRoot	主进程编号
hcclComm	hccl通信域接口获取的地址指针

Operation输入/输出

参数	类型	维度	类型	格式	描述
x	InTensor	[-1,...,-1]	float16/float/int8/int32/int64	ND	

5.7 atb/utils.h

5.7.1 Utils

函数功能

Utils接口类

函数原型

```
class Utils {  
public:  
    static Status GetVersion(int32_t *majorVersion, int32_t *minorVersion, int32_t *patchVersion);  
    static uint64_t GetTensorSize(const Tensor &tensor);  
    static uint64_t GetTensorSize(const TensorDesc &tensorDesc);  
    static uint64_t GetTensorNumel(const Tensor &tensor);  
    static uint64_t GetTensorNumel(const TensorDesc &tensorDesc);  
};
```

成员

成员名称	描述
GetVersion	返回加速库版本信息
GetTensorSize	返回Tensor对象的数据存储大小
GetTensorNumel	返回Tensor对象的数据个数